

Execution & Reconciliation Diagnostic

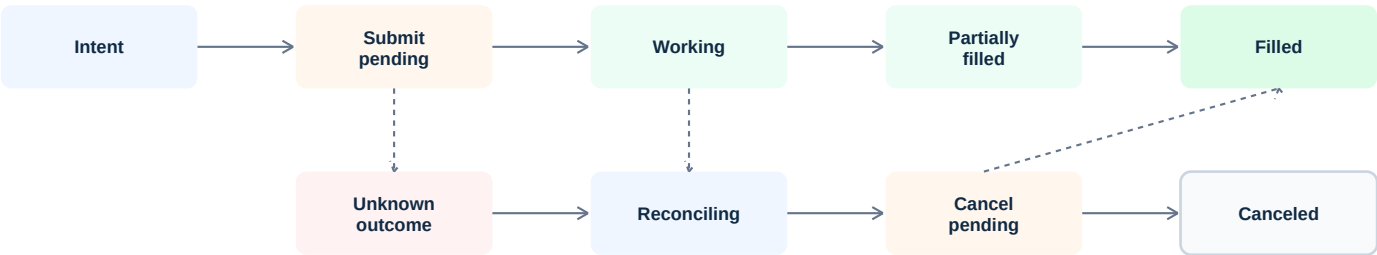
Sample deliverable for a one-day review of a Rust trading execution service

Synthetic public sample. The service, code paths, identifiers, events, and findings are fictional. Nothing here describes a real employer, repository, account, venue integration, strategy, incident, or financial performance.

Diagnostic boundary

The fictional system combines an asynchronous Rust executor, REST order API, WebSocket user stream, persistent local order store, and restart-time reconciliation. The review covers execution correctness and operational recovery - not security, alpha, profitability, or latency leadership.

State-transition map



Dashed transitions are ambiguity or recovery paths; routing stays gated while reconciling.

Core invariants: filled quantity is monotonic; one intent maps to at most one live venue order; timeout means unknown outcome, not rejection; routing remains paused until reconciliation completes.

Three high-risk failure paths

F-01 Lost ACK / timeout

Trigger

Venue accepts coid-42; the response is lost; generic retry submits again.

Impact

Duplicated exposure and an order local state may not own.

Control

Stable client ID, UnknownOutcome, query-before-resubmit.

F-02 Fill during cancel

Trigger

A partial fill arrives after cancel request but before cancel processing completes.

Impact

Wrong position, residual exposure, unsafe follow-up order.

Control

CancelPending, monotonic fill merge, authoritative snapshot.

F-03 Reconnect drift

Trigger

Stream reconnects after a sequence gap; routing resumes against stale memory.

Impact

Decisions use stale orders, fills, or available balance.

Control

Gap detection, snapshot barrier, event buffer, readiness gate.

From incident shape to testable patch plan

The real diagnostic replaces every illustrative path below with exact files, functions, commits, logs, and tests.

Minimal deterministic reproduction

Fixture: paused Tokio time, scripted transport, fake venue state, persistent in-memory store, and a user stream with explicit sequence numbers. The sequence below reproduces all three failure boundaries without network timing or live credentials.

1. Venue stores coid-42; submit response times out.
 2. Executor handles timeout and attempts recovery.
 3. Assert: exactly one venue order exists for coid-42.
 4. Assert: local state remains UnknownOutcome until query-by-client-id finds it.
 5. Deliver a 0.4 fill while cancel is pending; then return cancel success.
 6. Assert: filled_qty == 0.4 and canceled_qty == 0.6.
 7. Drop stream sequence 105; reconnect at 107 with a newer snapshot.
 8. Assert: routing stays paused until snapshot and buffered events converge.

Prioritized findings

Priority	Failure / impact	Illustrative code reference	Required change
P0	Ambiguous submit retry can duplicate exposure.	src/execution/submit.rs ::send_with_retry	Stable client ID and query-before-resubmit.
P0	Cancel path can discard a concurrent partial fill.	src/orders/reducer.rs ::apply_cancel	One reducer, monotonic fill merge, CancelPending.
P1	Reconnect resumes before venue state is restored.	src/stream/reconnect.rs ::on_open	Snapshot barrier, buffer, dedupe, readiness gate.
P2	Incidents lack order-level recovery telemetry.	src/telemetry/orders.rs	Correlation ID, state age, sequence-gap and reconcile metrics.

Patch and test plan

- 1

Make ambiguous submit explicit. Reuse one client order ID; query before any second submit. **Acceptance:** a lost response after venue acceptance creates one external order.
- 2

Centralize order-state reduction. Apply snapshots and events through one reducer with monotonic fill and terminal-state invariants. **Acceptance:** fill/cancel permutations converge.
- 3

Gate reconnect recovery. Pause routing, snapshot, buffer and deduplicate, reconcile, then resume. **Acceptance:** a sequence gap cannot release a new order early.
- 4

Lock behavior and expose drift. Add fake-clock, scripted-transport, restart, and reordering tests plus unknown-outcome, sequence-gap, reconcile-duration, and stale-state metrics.

One-day handoff: state and failure map; one deterministic reproduction where feasible; prioritized correctness and operational risks with exact code references; concrete patch/test plan; optional implementation quote after repository access and acceptance criteria are understood.